

END-TERM PROJECT- 02

Practical Programming with C (CSE 3544)

Publish on: 15-12-2025
Course Outcome: CO_{3...5}

Program Outcome: PO_{1...4}

Submission on: 08-01-2025
Learning Level: L₅

Building a Simple Shell

A *shell* is a process that does command-line interpretation. In the simplest case, the shell reads a command and forks a child to execute the command. The parent then waits for the child to complete before reading in another command. A real shell handles process pipelines, redirection and many more.

In this project, students are required to implement a shell (command interpreter) in C with limited capabilities. Use the system calls `fork()`, `exec()` family of functions, and `pipe()`. You also need familiarity with file descriptors and the associated functions `open()`, `close()`, `dup()`, and `dup2()`.

Part-1. Implement execution of simple commands. Observe the sample output of few shell commands.

```
[sanjaya]$ ls
raceprocess.c  raceprocessorder.c~  semshmwrite.c  shmreadrace.c~
semshmread.c  semshmwrite.c~       shmwriterace.c  solprocessorder.c~
semshmread.c~ shmreadrace.c        shmwriterace.c~ solraceproceSem.c
[sanjaya]$ who
sanjaya  :0                2019-04-01 16:45 (:0)
sanjaya  pts/0            2019-04-01 16:49 (:0.0)
sanjaya  pts/14            2019-04-01 17:08 (:0)
[sanjaya] $ date
Mon Apr  1 17:16:11 IST 2019
[sanjaya]$
```

Part-2. Your program must allow command-line arguments as given the sample cases.

```
[sanjaya]$ ls -l
total 88
-rwxrwxr-x 1 sanjaya sanjaya 13383 Mar 16 16:29 a.out
-rw-rw-r-- 1 sanjaya sanjaya  330 Mar 16 16:29 countingsem.c
-rw-rw-r-- 1 sanjaya sanjaya  101 Mar 16 16:24 ffd.c
-rw-rw-r-- 1 sanjaya sanjaya  670 Mar 14 18:55 raceprocess.c
-rw-rw-r-- 1 sanjaya sanjaya  659 Mar 14 18:52 raceprocess.c~
-rw-rw-r-- 1 sanjaya sanjaya  489 Mar 16 16:23 raceprocessorder.c
-rw-rw-r-- 1 sanjaya sanjaya  489 Mar 16 16:16 raceprocessorder.c~
-rw-rw-r-- 1 sanjaya sanjaya  637 Mar 14 19:12 semshmread.c
-rw-rw-r-- 1 sanjaya sanjaya  635 Mar 14 19:12 semshmread.c~
-rw-rw-r-- 1 sanjaya sanjaya  663 Mar 14 19:14 semshmwrite.c
-rw-rw-r-- 1 sanjaya sanjaya  651 Mar 14 19:12 semshmwrite.c~
-rw-rw-r-- 1 sanjaya sanjaya  403 Mar 14 19:06 shmreadrace.c
-rw-rw-r-- 1 sanjaya sanjaya  387 Mar 14 19:04 shmreadrace.c~
```

```
-rw-rw-r-- 1 sanjaya sanjaya 1687 Mar 14 14:59 smsolprocessorder.c
-rw-rw-r-- 1 sanjaya sanjaya 1697 Mar 14 14:59 smsolprocessorder.c~
-rw-rw-r-- 1 sanjaya sanjaya 1073 Mar 14 12:25 solraceprocessSem.c
-rw-rw-r-- 1 sanjaya sanjaya 1072 Mar 14 12:25 solraceprocessSem.c~
[sanjaya]$ ls -l /usr/local
total 36
drwxr-xr-x 2 root root 4096 Feb 20 2016 bin
drwxr-xr-x 2 root root 4096 Aug 5 2015 etc
drwxr-xr-x 2 root root 4096 Aug 5 2015 games
drwxr-xr-x 2 root root 4096 Jan 18 2016 include
drwxr-xr-x 6 root root 4096 Feb 17 2017 lib
lrwxrwxrwx 1 root root 9 Dec 28 2015 man → share/man
drwxr-xr-x 4 root root 4096 Feb 20 2016 MATLAB
drwxr-xr-x 2 root root 4096 Aug 5 2015 sbin
drwxr-xr-x 9 root root 4096 Jan 18 2016 share
drwxr-xr-x 2 root root 4096 Aug 5 2015 src
[sanjaya]$
```

Part-3. Your program must be equipped to handle pipe and cascading pipe commands.

```
[sanjaya]$ who | grep sanjaya
sanjaya :0 2019-04-01 16:45 (:0)
sanjaya pts/0 2019-04-01 16:49 (:0.0)
sanjaya pts/14 2019-04-01 17:08 (:0)
[sanjaya]$ man ls | grep ls | wc -l
11
[sanjaya]$ ls -l /usr/local | wc
11 94 484
```

Part-4. Most shells allow redirection of standard input, standard output and possibly standard error from the command line. **Filters** are programs that read from standard input and write to standard output. Redirection on the command line allows filters to operate on other files without recompilation. So, your program must meet the input-output redirection capabilities.

Example 1 you know the `cat` command with no command-line arguments reads from standard input and echoes to standard output. The command `cat > sample.txt` redirects standard output to `sample.txt` with `>`.

Example 2 The command `cat < sample.txt` redirects its standard input to `sample.txt` and displays the content of the file.

Example 3 The command `cat sample.txt > sample-1.txt` redirects the contents of the file `sample.txt` to the file `sample-1.txt`.

Example 4 The given command `cat < myinput.txt > myoutput.txt` redirects its standard input to `myinput.txt` and its standard output to `myoutput.txt`.

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific outcomes (PSOs)

COs	POs											PSOs	
	1	2	3	4	5	6	7	8	9	10	11	1	2
CO 1													
CO 2													
CO 3	3	3	3									2	2
CO 4	3	3	3	2								2	2
CO 5	3	3	3	2								2	2
CO 6													